PukSquash Developer Manual

Administrator Court Management

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

Court management maintains the physical courts offered in the availability grid. An ACTIVE ADMIN can create a court, change its name or description, change its ACTIVE/INACTIVE status, or request deletion. An inactive court must not be offered for new bookings.

ManageCourts calls the protected admin endpoints. Laravel validates submitted fields and uses CourtRepository to run the corresponding Data Connect operation. Changes are reread where appropriate, and the user-facing availability grid must be refreshed so new court status is visible.

1 1 Files and Responsibilities

resources/js/pages/ManageCourtsPage.vue — administrative court interface

app/Http/Controllers/Api/AdminCourtController.php — court CRUD endpoints

app/Repositories/CourtRepository.php — queries, mutations and cache handling

dataconnect/schema/schema.gql — Court type

dataconnect/example/mutations.gql — CreateCourt, UpdateCourt and DeleteCourt

1 2 API Contract

GET /api/admin/courts lists courts. POST creates one and returns 201. PUT /api/admin/courts/{id} updates an existing court. DELETE /api/admin/courts/{id} returns 204 on successful deletion. Public feature reads still use authenticated /api/courts endpoints.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Creating a court

app/Http/Controllers/Api/AdminCourtController.php | source line 25 onward; blank and comment lines omitted

```
public function store(Request $request): JsonResponse
{
    $validated = $request->validate([
        'name'     => 'required|string|max:255',
        'description' => 'sometimes|string|max:255',
        'status'   => 'sometimes|in:ACTIVE,INACTIVE',
    ]);
    $this->courts->create($validated);
    return response()->json(['message' => 'Court created successfully.'], 201);
}
```

Explanation

The controller accepts a required court name and optional description and status. Request validation restricts field lengths and the status vocabulary. CourtRepository provides defaults and executes CreateCourt; the endpoint responds with a success message rather than a complete court object.

Physical Code Explanation

1 name must be a string no longer than 255 characters.

2 description is optional but, when supplied to store, must be a string no longer than 255 characters.

3 status is optional and must be ACTIVE or INACTIVE.

4 The repository handles persistence; response()->json sends HTTP 201 so the frontend can reload the court list.

For an update, the controller verifies the court exists, validates optional fields, and chooses UpdateCourt or the status-only operation. Missing courts return 404 and requests with no recognised changes return 422.

3 Court schema and identity

dataconnect/schema/schema.gql | source line 57 onward; blank and comment lines omitted

```
type Court @table {
  name: String! @unique
  description: String
  status: String! @default(value: "ACTIVE")
  createdAt: Timestamp! @default(expr: "request.time")
  updatedAt: Timestamp! @default(expr: "request.time")
}
```

Explanation

The Court entity has a unique name, optional description and status defaulting to ACTIVE. Bookings reference its generated application key. Deactivation preserves those references while preventing new selection of unused inactive cells.

Physical Code Explanation

1 name is non-null and unique. A duplicate name is a database conflict, even if the form's string validation passes.

2 description is nullable in the schema. The update controller permits null, while the creation controller expects a string if supplied.

3 status is a string with an ACTIVE default. The allowed vocabulary is constrained by request validation, not an enum declaration in this schema.

4 createdAt and updatedAt have request-time defaults; update timestamps must be explicitly maintained where required by the chosen mutation.

Deleting a court can fail when reservations reference it. The baseline DeleteCourt path does not implement an archive workflow or a custom conflict response. Treat deactivation as the normal operational option for temporarily unavailable courts.

4 Verification and Maintenance

Verification checklist

1 Create a new court with and without optional fields. Confirm its appearance in the admin list and the availability grid.

2 Rename a court and verify natural ordering, including names containing 2 and 10.

3 Deactivate a court and confirm a fresh booking request for it returns 422 even if the browser had an older grid snapshot.

4 Try an unknown ID, over-length name, duplicate name and invalid status. Verify controlled UI feedback.

5 Delete a disposable court without bookings and confirm 204 is handled without JSON parsing. Test the referenced-court case separately.

Implementation boundaries

There is no complete administrator time-slot maintenance endpoint in this Firebase baseline. Court management must not be documented as providing time-slot CRUD.

The later admin dashboard statistics are not part of the documented Firebase baseline. Implement equivalent Data Connect queries before documenting live counts or utilisation as supported Firebase behaviour.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).